Clase D02 · Patrones GRASP

Bloque 2 · Diseño · Clase 2 de 9

Modelación y Diseño de Sistemas · 2026 · Bloque Diseño

Prof. Ing. Fernando Bono · ESCMB UNC · Pregrado AUSI

Agenda

- Recap D01: alta cohesión + bajo acoplamiento
- El problema: ¿cómo decidir quién hace qué?
- ¿Qué son los patrones GRASP?
- Catálogo: 9 patrones (5 básicos + 4 avanzados)
- Los 5 GRASP básicos:
- Experto (Information Expert)
- Creador (Creator)
- Bajo Acoplamiento
- Alta Cohesión
- Controlador
- Los 4 GRASP avanzados:
- Polimorfismo
- Fabricación Pura
- Indirección
- Law of Demeter (No hables con extraños)
- Aplicación integrada al UC "Procesar venta"
- Conexión con SOLID
- Errores comunes
- Cierre + qué viene (D03 GoF)

1 · Recap D01 + el problema

En D01 cerramos con la regla de oro:

- Alta cohesión: cada clase hace UNA cosa, bien.
- Bajo acoplamiento: las clases se comunican por interfaces, no por internals.

El problema concreto

Cuando llega el UC "Procesar venta", aparece la pregunta:

- ¿Quién calcula el total? ¿Venta, GestorVenta o una nueva Calculadora?
- ¿Quién crea un Item? ¿La Venta o el Producto?
- ¿Quién valida el stock? ¿Producto, Venta o RepositorioStock?
- ¿Quién recibe el evento del cajero?

Si la asignación es arbitraria, el código termina frágil y acoplado.

¿Pero cómo se decide a qué clase asignar cada responsabilidad?

2 · ¿Qué son los patrones GRASP?

General Responsibility Assignment Software Patterns.

- Propuestos por Craig Larman en UML y Patrones (cap. 17-22).
- No son patrones de código (no se "copia y pega"): son principios de razonamiento.
- Sirven para decidir dónde poner cada método/responsabilidad.
- Trabajan al nivel del diseño OO, antes que el código.
- Son complementarios entre sí y con los principios de D01.

GRASP vs GoF

	GRASP (Larman)	GoF (Gamma et al.)
Foco	Asignar responsabilidades	Resolver problemas concretos de diseño
Nivel	Conceptual (decidir QUIÉN hace QUÉ)	Estructural (CÓMO se implementa)
Cuándo	Durante el diseño OO	Durante el diseño detallado
Cantidad	9 patrones	23 patrones
Ejemplo	Experto, Controlador	Singleton, Observer, Strategy

GRASP es la base. GoF (clase D03) viene después.

3 · Catálogo de los 9 patrones GRASP

Los 5 primeros son los fundamentales. Los 4 últimos son avanzados y muchas veces emergen como aplicación de los primeros.

#	Patrón	Pregunta que responde
1	Experto (Information Expert)	¿Quién tiene la información para hacerlo?
2	Creador (Creator)	¿Quién debe instanciar este objeto?
3	Bajo Acoplamiento	¿Cómo minimizo dependencias entre clases?
4	Alta Cohesión	¿Cómo mantengo a cada clase enfocada?
5	Controlador	¿Quién recibe los eventos del sistema?
6	Polimorfismo	¿Cómo manejo variantes de comportamiento?
7	Fabricación Pura	¿Y si no hay clase de dominio que sirva?
8	Indirección	¿Cómo evito el acoplamiento directo?
9	Law of Demeter (No hables con extraños)	¿Con quién puedo hablar?

4 · GRASP 1 · Experto (Information Expert)

Por qué

- Evita pedir datos a otro objeto para decidir afuera (rompe encapsulación).
- Promueve alta cohesión y bajo acoplamiento.
- Mantiene los datos y el comportamiento juntos (principio OO básico).

Ejemplo ChangoMás · ¿quién calcula el total de la venta?

MAL (decidir afuera):

GestorVenta accede a los items y precios de Venta. Acoplamiento alto.

BIEN (la experta es Venta):

Venta conoce sus items → Venta es la experta en calcular el total.

```
class GestorVenta:
    calcularTotal(venta):
        total = 0
        for item in venta.items:
            total += item.precio * item.cantidad
        return total
```

Asigná la responsabilidad a la clase que tiene la información necesaria para cumplirla.

5 · GRASP 2 · Creador (Creator)

Por qué

- Mantiene el bajo acoplamiento (quien usa el objeto suele saber crearlo).
- Evita la dispersión de new por todo el código.
- Refleja relaciones del dominio.

Ejemplo ChangoMás · ¿quién crea un Item?

Venta agrega ítems (composición) y tiene los datos para inicializarlos → Venta es la creadora.

Otros candidatos en ChangoMás

- Venta crea Item (composición).
- GestorVenta crea Venta (la controla y la inicializa).
- RepositorioStock crea instancias de Stock al hidratarlas de BD.

```
class Venta:
    agregarItem(producto: Producto, cantidad: int): void
        item = Item(producto, cantidad)      # ← Venta crea Item
        self.items.add(item)
```

La clase B debería crear a la clase A si... B contiene a A, agrega a A, registra a A, usa intensivamente a A, o tiene los datos para inicializarla.

Síntomas de alto acoplamiento

- Cambiar una clase obliga a tocar varias.
- Los tests requieren mockear medio mundo.
- Las clases conocen detalles internos de otras (campos privados, formatos, protocolos).

Ejemplo ChangoMás · notificación al cliente

MAL — GestorVenta conoce el protocolo de email:

GestorVenta está acoplada al SMTP, al servidor, a las credenciales.

BIEN — delegar a una interfaz:

Mañana el notificador puede ser SendGrid, WhatsApp o un mock para test. GestorVenta ni se entera.

```
class GestorVenta:
    cerrarVenta():
        smtp = SMTP('mail.changomas.com.ar', 587)
        smtp.login(USER, PASS)
        smtp.sendmail(cliente.email, 'Comprobante', html)
```

Asigná las responsabilidades de modo que minimices el acoplamiento entre clases.

Síntomas de baja cohesión

- La clase tiene un nombre genérico (Manager, Utils, Helper).
- El nombre necesita "And" (UsuarioYPedidoManager).
- Tiene métodos de áreas no relacionadas (BD + email + cálculos + UI).
- Difícil de testear porque mezcla responsabilidades.

Ejemplo ChangoMás · split de una God Class

MAL: GestorCaja que hace todo.

BIEN: cada cosa en su clase.

```
class GestorCaja:
    procesarVenta()
    enviarComprobantePorEmail()
    generarReporteDiario()
    conectarBaseDeDatos()
    validarDNI()
    consultarStock()
```

Asigná responsabilidades de modo que cada clase tenga responsabilidades estrechamente relacionadas.

8 · GRASP 5 · Controlador

Por qué

- Centraliza el punto de entrada de cada UC.
- Separa la capa de UI del dominio.
- Evita que la UI conozca la lógica de negocio.

Las 2 variantes

- Controlador de fachada (1 por sistema/subsistema): ej. SistemaPOS.
- Controlador de caso de uso (1 por UC): ej. GestorVenta, GestorDevolucion. ← Más usado en sistemas medianos/grandes.

Ejemplo ChangoMás

La UI del cajero llama a GestorVenta.escanear(codigo) — no sabe nada de Producto, Stock ni Comprobante.

Anti-pattern: Bloated Controller

```
class GestorVenta:          # controlador del UC "Procesar venta"
    iniciar(cajero)
    escanear(codigo)
    cerrarVenta(formaPago)
```

Asigná la responsabilidad de manejar un evento del sistema a una clase que represente uno de:

9 · Resumen · 5 GRASP básicos

Patrón	Pregunta	Ejemplo ChangoMás
Experto	¿Quién tiene los datos?	Venta.calcularTotal()
Creador	¿Quién debe instanciar?	Venta crea Item
Bajo Acoplamiento	¿Cómo minimizo dependencias?	Notificador interfaz
Alta Cohesión	¿Cómo mantengo enfoque?	Split de GestorCaja
Controlador	¿Quién recibe el evento?	GestorVenta para UC venta

Estos 5 cubren ~80% de las decisiones de diseño OO. Los 4 que vienen son refinamientos.

10 · GRASP 6 · Polimorfismo

Por qué

- Evita if/switch por tipo (anti-pattern: Type Check).
- Extensible: agregar un nuevo tipo no rompe el código existente (Open/Closed).
- Cada variante sabe su propio comportamiento.

Ejemplo ChangoMás · medios de pago

MAL — if/else por tipo:

Agregar un nuevo medio = tocar GestorVenta.

BIEN — polimorfismo:

Agregar PagoCripto = crear una clase nueva. Cero cambios en GestorVenta.

```
class GestorVenta:
    cobrar(venta, tipo):
        if tipo == 'EFECTIVO':
            ...lógica de caja...
        elif tipo == 'TARJETA':
            ...lógica de POSnet...
        elif tipo == 'QR':
            ...lógica de Mercado Pago...
```

Cuando un comportamiento varía según el tipo, asignalo a una jerarquía de clases usando una operación polimórfica.

11 · GRASP 7 · Fabricación Pura (Pure Fabrication)

Por qué

- A veces poner la responsabilidad en una clase del dominio rompe su cohesión.
- O genera acoplamiento con cosas técnicas (BD, red, framework).
- Crear una clase "artificial" mantiene el dominio limpio.

Ejemplo ChangoMás · ¿quién persiste la venta?

MAL — la propia Venta se persiste:

Venta ahora conoce JDBC, SQL, esquemas. Su cohesión se rompió.

BIEN — Pure Fabrication: RepositorioVenta

Venta queda limpia. La persistencia es responsabilidad de una clase técnica creada para eso.

```
class Venta:
    guardar():
        conn = JDBC.getConnection(...)
        stmt = conn.prepareStatement('INSERT INTO ventas ...')
        ...
```

Cuando ningún concepto del dominio sirve como casa natural de una responsabilidad, inventá una clase nueva que no represente un concepto del dominio.

12 · GRASP 8 · Indirección

Por qué

- Reduce dependencias entre módulos que no necesitan conocerse.
- Permite cambiar el lado A sin afectar al B.
- Facilita reutilización y testing.

Ejemplo ChangoMás · facturación electrónica con AFIP

MAL — Venta habla directo con AFIP:

Si AFIP cambia el protocolo, mañana hay que tocar Venta.

BIEN — clase intermediaria: ServicioFiscal

Si AFIP cambia, solo se toca ServicioFiscal. Venta ni se entera.

```
class Venta:
    facturar():
        afipClient = WebServiceAFIP('https://wsfe.afip.gob.ar/...')
        cae = afipClient.solicitarCAE(...)
```

Para evitar el acoplamiento directo entre dos clases, ponelas a comunicar a través de un objeto intermediario.

13 · GRASP 9 · Law of Demeter (No hables con extraños)

Por qué

- No invocar métodos sobre el retorno de otro método ("tren de getters").
- Evita el efecto cascada: si cambia algún eslabón intermedio, se rompe todo.
- Promueve interfaces de "Tell, don't ask".

Ejemplo ChangoMás · obtener calle de la dirección de entrega

MAL — tren de getters:

Si Cliente.getDireccion() ahora devuelve un Optional<Direccion> o cambia su tipo, esto se rompe en todos los callers.

BIEN — preguntale directo a Venta (delegación):

Internamente, Venta resuelve el detalle. Los callers no se enteran.

Cuándo NO aplicarlo

- En DTOs / value objects planos no aporta (son por definición transparentes).
- En fluent APIs (query.from(...).where(...).orderBy(...)) está bien encadenar.

```
calle = venta.getCliente().getDireccion().getCalle()
```

Un método solo puede invocar métodos de:

14 · Resumen · 4 GRASP avanzados

Patrón	Cuándo aplicarlo	Ejemplo ChangoMás
Polimorfismo	Comportamiento que varía por tipo	MedioPago con cobrar()
Fabricación Pura	Falta clase de dominio natural	RepositorioVenta
Indirección	Evitar acoplamiento directo	ServicioFiscal ↔ AFIP
Law of Demeter	Evitar trenes de getters	venta.obtenerCalleEntrega()

15 · Aplicación integrada · UC "Procesar venta"

Cada paso del UC aplica uno o más GRASP:

Paso del UC	GRASP aplicado	Cómo se resuelve
Cajero escanea producto	Controlador	GestorVenta.escanear(codigo)
Buscar producto por código	Experto	RepositorioProducto.obtener(codigo) (Pure Fabrication)
Agregar item a la venta	Creador	Venta.agregarItem(producto, 1) crea Item
Calcular subtotal del item	Experto	Item.subtotal() (tiene precio y cantidad)
Calcular total de la venta	Experto	Venta.calcularTotal()
Cobrar según forma de pago	Polimorfismo	formaPago.cobrar(total)
Emitir comprobante fiscal	Indirección	ServicioFiscal.emitir(venta)
Descontar stock	Fabricación Pura + Experto	RepositorioStock.descontar(items)
Notificar al cliente	Bajo Acoplamiento	Notificador.enviar(comprobante)
Cerrar la venta	Alta Cohesión	Venta.cerrar() solo cambia su estado

Todo el UC se resolvió con GRASP. Sin GoF todavía. Esa es la potencia.

16 · Conexión con SOLID

GRASP y SOLID no compiten, se complementan:

GRASP	SOLID relacionado	Cómo se conectan
Alta Cohesión	Single Responsibility	Misma idea, distinto nivel: GRASP en la clase, SRP en el cambio.
Polimorfismo	Open/Closed + Liskov	Polimorfismo es el mecanismo; OCP es la propiedad.
Bajo Acoplamiento	Dependency Inversion	DIP es bajo acoplamiento llevado a la dependencia entre módulos.
Indirección	Interface Segregation	La interfaz que media debe ser fina y específica.
Controlador	SRP	Cada UC, su controlador. Cambio del UC = cambio de su controlador.

GRASP responde dónde poner la responsabilidad. SOLID responde cómo estructurar los módulos para que cambien bien.

17 · Errores comunes al aplicar GRASP

- Confundir Controlador con todo lo que orquesta → Bloated Controller (God Class disfrazada).
- Aplicar Pure Fabrication sin necesidad → sumar clases técnicas que solo confunden.
- Romper Law of Demeter sin notarlo → trenes de getters en logging, mappers, vistas.
- Usar Experto mal: pedirle el dato a uno y decidir afuera (rompe el patrón).
- Saltarse Polimorfismo y volver a if-else por tipo cuando aparece una variante nueva.
- Pensar que GRASP "es para el examen" — son la base del diseño OO real.

No memorices los nombres. Internalizá las preguntas que respondés con cada patrón.

18 · Cierre + qué viene

Lo que vimos hoy

- 9 patrones GRASP para asignar responsabilidades sistemáticamente.
- Los 5 básicos (Experto, Creador, Bajo Acoplamiento, Alta Cohesión, Controlador) cubren la mayoría de decisiones de diseño.
- Los 4 avanzados (Polimorfismo, Pure Fabrication, Indirección, Law of Demeter) refinan casos puntuales.
- Cada paso del UC "Procesar venta" se resolvió con GRASP — sin necesidad de GoF.
- GRASP y SOLID son la misma idea desde dos ángulos.

Próximo: D03 · Patrones GoF

GoF nos dará soluciones concretas a problemas recurrentes:

- Singleton (cuando hay UNA sola instancia legítima).
- Observer (cuando varios objetos reaccionan al cambio de uno).
- Strategy (cuando un algoritmo cambia en runtime).
- Factory Method + Abstract Factory (cuando la creación es compleja).
- Decorator, Adapter, Composite, State + variantes.

Bibliografía

- Larman, C. — UML y Patrones (2da ed.), caps. 17, 18, 22.

Próxima clase (D03): Patrones GoF aplicados al caso ChangoMás.